Fernando Alves da Silva
Leonardo Felix Gajardo

Análise comparativa de algoritmos baseados em busca local para resolução do Problema do Ladrão Viajante

São Paulo - Brasil

2020

Fernando Alves da Silva
Leonardo Felix Gajardo

Análise comparativa de algoritmos baseados em busca local para resolução do Problema do Ladrão Viajante

Monografia apresentada na disciplina Trabalho de Conclusão de Curso, como parte dos requisitos para obtenção do título de Bacharel em Ciência da Computação.

Centro Universitário Senac - Santo Amaro

Bacharelado em Ciência da Computação

Orientador: Leonardo Takuno

São Paulo - Brasil

2020

Lista de ilustrações

Figura 1 – Exemplo de uma instância para TTP_1. Fonte: The travelling thief problem: The first step in the transition from theoretical problems to realistic problems(BONYADI; MICHALEWICZ; BARONE, 2013). . . 18
Figura 2 – Exemplo de *crossover* com um ponto de corte. Fonte: Utilização de algoritmos genéticos para otimização de soluções para o timetabling escolar.(SIMãO, 2013). 21
Figura 3 – Demonstrativo de máximo global e máximos locais. Fonte: Algoritmos evolutivos aplicados a síntese e otimização de sistemas de ancoragem(ALBRECHT et al., 2019) 26

Resumo

Este trabalho busca estudar o problema do ladrão viajante, compreender o que levou sua formulação, o estado da arte do problema com soluções propostas ao longo do tempo, aplicar os algoritmos de MA2B, CS2SA e CS2B para resolução e com base nestas aplicações realizar uma análise comparativa sobre o tempo de processamento, qualidade da solução e o quanto cada configuração de entrada pode afetar a solução.

Muito além das comparações, este trabalho de pesquisa busca a partir dos resultados obtidos compreender a viabilidade da aplicação dos algoritmos para problemas reais levando em consideração o tempo de execução e qualidade da solução.

Palavras-chaves: TTP, Problema do Ladrão Viajante, análise comparativa, *Hill Climbing*, *Simulated Annealig*, Meta-heurística, *bit-flip*, *algoritmo evolutivo*.

Lista de abreviaturas e siglas

AEs	Algoritmos Evolucionários
AM	Algoritmo Memético
CE	Computação Evolucionária
CS2B	CoSolver-based with 2-OPT and Bit-flip
CS2SA	CoSolver-based with 2-OPT and Simulated Annealing
HC	Hill Climbing
MA2B	Memetic Algorithm with 2-OPT and Bit-flip
MPX	Multiple-point crossover
NP	Non-Polynomial
P	Polynomial
PM	Problema da Mochila
PCV	Problema do Caixeiro Viajante
PLV	Problema do Ladrão Viajante
SA	Simulated Annealing
TTP	Travelling Thief Problem

Sumário

Lista de ilustrações **2**

1 INTRODUÇÃO **7**

2 OBJETIVOS **9**
2.1 Objetivos Específicos **9**

3 CONCEITOS GERAIS **10**
3.1 Complexidade Computacional **10**
3.1.1 Classe P 11
3.1.2 Classe NP 11
3.1.3 Classe NP-Completo 12
3.1.4 Classe NP-Difícil 12
3.2 Problema da Mochila **13**
3.2.1 Programação Dinâmica 13
3.2.2 Variante binária (0-1) 14
3.3 Problema do Caixeiro Viajante **14**
3.4 Problema do Ladrão Viajante **15**
3.4.1 Razão de formulação do problema 15
3.4.2 Subproblemas 16
3.4.2.1 Definição do Problema do Caixeiro Viajante para o Problema do Ladrão Viajante 16
3.4.2.2 Definição do Problema da Mochila Binária para o Problema do Ladrão Viajante . 16
3.4.3 Definição do Problema do Ladrão Viajante 17
3.4.4 Modelos do problema 17
3.4.4.1 Modelo TTP_1 17
3.5 Busca Local **19**
3.5.1 2-OPT 19
3.6 Algoritmos gulosos **19**
3.7 Computação Evolucionária **20**
3.7.1 Algoritmos Evolucionários 20
3.7.1.1 Algoritmos Meméticos 22
3.8 Estado da arte do Problema do Ladrão Viajante **23**
3.8.1 Heurísticas de solução única 23
3.8.2 Aproximação Exata 24
3.9 Hill Climbing **24**
3.9.1 Problemas e Limitações 25

3.9.2 Variantes . . . 26
3.9.2.1 Steepest Ascent . . . 26
3.9.2.2 Estocástico . . . 27
3.9.2.3 Reinicialização Aleatória . . . 27
3.10 Simulated Annealing . . . 28
3.10.1 Problemas e Limitações . . . 29
3.11 CoSolvers para a interdependência do PLV . . . 29
3.12 MA2B . . . 30
3.13 Algoritmos baseados em *CoSolvers* . . . 32
3.13.1 CS2SA . . . 32
3.13.2 CS2B . . . 32

4 METODOLOGIA . . . 33
4.1 Configuração Experimental . . . 33
4.2 Solução Inicial . . . 34
4.3 Algoritmos Base . . . 34
4.3.1 *2-OPT steepest ascent hill climbing* . . . 34
4.3.2 *Bit-flip* . . . 35
4.3.3 Simulated Annealing . . . 35

5 RESULTADOS . . . 37

6 CONCLUSÃO . . . 39

REFERÊNCIAS . . . 40

1 Introdução

Atualmente, o estudo de algoritmos de otimização tem aumentado consideravelmente, tanto pelas diversas aplicações quanto pela capacidade de processamento das máquinas ter aumentado notadamente, abrindo uma série de possibilidades que não existiam antes.

Esses estudos incluem desafios como o problema do caixeiro viajante, clique grande, coloração de grafos, problemas da mochila e diversos outros pertencentes a classe NP-Difícil, que embora possam ter métodos exatos para resolução, não podem ser resolvidos para instâncias muito grandes dada a quantidade de tempo necessário, além de serem impostas determinadas adaptações aos problemas quando imaginados em cenários reais.

Consideradas as deficiências para solucionar problemas NP-Difíceis, podemos considerar o estudo acerca de heurísticas extremamente relevante para essa classe de problemas, em razão de conseguirem uma solução boa em tempo hábil.

Embora heurísticas resolvam um dos problemas da classe NP-Difícil, sua especificidade sobre um problema acaba por dificultar aplicações em problemas palpáveis, acarretando na necessidade de algoritmos mais genéricos, que possibilitem a adaptação a problemas com uma baixa quantidade de alterações, denominados como meta-heurísticas.

Apoiado na importância dessa classe de problemas, verificou-se que estudos para o problema do ladrão viajante tem tido vital relevância, uma vez que é uma combinação do problema da mochila e do problema do caixeiro viajante, que são facilmente visualizados em questões de logística no mundo real, e por ter sido formulado há pouco, não só a aplicação de meta-heurísticas para sua resolução é importante , como também se faz necessária a comparação destas para uma melhor compreensão de sua utilização em uma série de variantes possíveis.

Levando em consideração os pontos citados, acredita-se que o estudo e comparação de meta-heurísticas que resolvem o problema do ladrão viajante possa ser uma questão expressamente válida a ser estudada, uma vez que a verificação dos pesos utilizados e tempo de processamento possam ser úteis de acordo com o tipo de aplicação.

Espera-se conseguir analisar os resultados dos algoritmos CS2SA, CS2B e MA2B em diferentes tipos de instâncias de teste, dado que todos operam com algoritmos clássicos de busca local utilizando as meta-heurísticas de *Hill Climbing*, *Simulated Annealing* e técnica *bit-flip*.

A apresentação de novas vias para o problema do ladrão viajante representa um grande auxílio na comunidade que está a estudar o problema, sendo que pequenas alterações

em cada tipo de resolução podem servir de base significativa para aplicações futuras do problema.

2 Objetivos

O principal objetivo deste trabalho é realizar um estudo comparativo entre diferentes tipos de algoritmos que utilizam de busca local aplicados para resolução do PLV.

2.1 Objetivos Específicos

No que diz respeito aos objetivos específicos, pretende-se:

- Apresentar uma visão geral sobre PLV e seus subproblemas.
- Implementar diferentes algoritmos para resolução do PLV, realizar uma coleta de dados e realizar uma comparação dos resultados obtidos, compreendendo os melhores cenários para cada aplicação.
- Descrever os algoritmos utilizados e todo o processo de obtenção e análise das métricas coletadas a partir do código fonte.

3 Revisão Bibliográfica

Este capítulo apresenta alguns algoritmos e conceitos que serão utilizados nos capítulos posteriores para observação e explicação da construção do Problema do Ladrão Viajante(ou PLV, tradução do nome original *Travelling Thief Problem*). Um aspecto importante do PLV é sua formação, que parte da união do Problema da Mochila(ou PM, tradução do nome original *knapsack problem*) e o Problema do Caixeiro Viajante(ou PCV, tradução do nome original *travelling salesman problem*). Portanto, para ter um bom entendimento de todas as fases aqui descritas, é importante compreender conceitos e conexões estabelecidos por meio do estudo de cada um destes.

Além de questões sobre os subproblemas compositores do PLV, buscamos realizar também estudos sobre os algoritmos baseados em busca local de *Hill Climbing*(HC) e *Simulated Annealing*(SA), além dos conceitos de Algoritmos Evolutivos(AEs), para que a viabilidade de aplicação para PLV utilizando estes seja compreendida e fundamentada.

3.1 Complexidade Computacional

A complexidade computacional é um ramo da ciência da computação que busca classificar problemas computacionais de acordo com sua dificuldade, verificando problemas de decisão e compreendendo como resolve-los, separando-os em diferentes tipos de problemas(SIPSER, 2005).

Após os computadores deixarem de ser tão restritos a algumas instituições científicas por volta da década de sessenta, a computação sofreu certa expansão em algumas áreas, o que desencadeou um maior interesse pelo estudo de algoritmos, uma vez que agora podiam ser testados com maior facilidade. Embora algoritmos que se aproximassem muito da melhor solução fossem conhecidos para resolução de determinados problemas, por muitas vezes não eram suficientes por questões de tempo de execução.

Surge então a teoria da complexidade computacional, necessária para a compreensão e classificação da dificuldade sobre determinados problemas, que foram subdivididos em classes e comparados entre si em base de determinadas características.

A partir da definição destas classes, antigos e atuais problemas da computação passam a ser classificados, e problemas novos podem ser comprovados de melhor maneira, seja por redução ou outras técnicas abrangidas por este ramo da computação, extremamente relevantes para a compreensão de certos comportamentos. Alguns destes problemas tem grande busca por resolução ainda hoje, tal como P. versus NP(OLIVEIRA, 2010).

Os problemas estudados pelo ramo da complexidade computacional utilizam de

uma Máquina de Turing para formalizar e comprovar soluções, já que a mesma é capaz de modelar qualquer máquina de computação. Uma Máquina de Turing é um modelo abstrato de computador que utiliza de aspectos lógicos como estados, transições e memória e independe de implantação física(SIPSER, 2005).

3.1.1 Classe P

A classe P refere-se ao conjunto de problemas que podem ser resolvidos em tempo polinomial por uma máquina de Turing determinística. Todos os problemas que se enquadram nessa classe podem ser resolvidos com um tempo de execução $O(n^{\mathrm{k}})$, onde k é constante(TURING, 1936)(SIPSER, 2005).

Pode-se definir a classe P como o conjunto de problemas solucionáveis para os computadores atuais, embora possam levar quantidades consideráveis de tempo.

Para P, temos algoritmos que recebem uma instância x e tem um tempo de execução polinomial em função de x. Caso x tenha solução, o algoritmo será executado e retornará essa solução, caso não tenha, o mesmo conseguirá identificar e retornará que não há solução.

Em P podemos encontrar diversas resoluções para problemas naturais, como o cálculo do Máximo Divisor Comum e o problema de caminho PATH.(SIPSER, 2005).

3.1.2 Classe NP

A classe NP se refere ao conjunto de problemas que são decidíveis em tempo polinomial por uma máquina de Turing não-determinística e que podem ter uma certificação verificada em tempo polinomial(TURING, 1936)(SIPSER, 2005).

Nesta classe se enquadram diversos problemas de busca e otimização que são frequentemente estudados ainda hoje, como o PCV, PM ou o problema da Satisfabilidade Booleana(SAT)(ZHANG et al., 2009)(WOEGINGER, 2003).

Máquinas de Turing não determinísticas são tomadas como irreais por conseguirem, a cada passo, executar infinitas instruções paralelamente, dado que se uma destas instruções chegar em um resultado o problema é resolvido.

Para a execução deste tipo de máquina necessitaríamos de um computador real com tal capacidade, o que é inexistente atualmente. Podemos então afirmar que algoritmos que buscam resultados exatos para esta classe de problemas terão que considerar todas estas instruções, resultando em um grande tempo de processamento.

Outra característica importante desta classe de problema é que verificar uma possível solução, comumente chamado de certificado, para algum problema deste conjunto

é factível em tempo polinomial. Vale notar que esta checagem se liga ao fato de ser uma solução válida, e não a melhor solução(SIPSER, 2005).

3.1.3 Classe NP-Completo

A classe NP-Completo é uma subclasse de NP tal que qualquer problema em NP pode reduzir-se a um problema NP-Completo. Podemos formalizar uma redução da seguinte forma: um problema X é polinomialmente redutível a um problema Y sse existe um algoritmo que consegue resolver X usando uma sub-rotina hipotética que consegue resolver Y, de modo que se a sub-rotina for polinomial o algoritmo para X também é polinomial(SIPSER, 2005). Sendo assim, uma vez que um problema NP-Completo pudesse ser resolvido em tempo factível(polinomial), qualquer problema em NP poderia ser resolvido utilizando este algoritmo(BALL, 1986).

Pode-se definir um problema NP-Completo como um problema $W \in$ NP, de maneira que todo problema em NP seja redutível a W em tempo polinomial. A prova de que $W \in$ NP pode ser demonstrada por meio da verificação de W em tempo polinomial(SIPSER, 2005).

SAT é reconhecido como o primeiro problema provado como NP-Completo, e busca solucionar se há uma valoração para as variáveis de uma fórmula booleana tal que essa valoração satisfaça essa fórmula, a tornando verdadeira(COOK, 1971). Pode-se observar aplicações de SAT para observar a equivalência de dois circuitos combinatórios ou relações inerentes a um circuito combinatório, onde sabe-se que um mapeamento direto pode ser estabelecido de um circuito combinatório para um tipo de fórmula SAT(FUX, 2004).

Posteriormente outros problemas foram identificados por Richard Karp, expandindo essa área de problemas e atraindo interesse para futuras descobertas de problemas dessa classe.

Ainda não há provas conclusivas de que problemas da classe de NP-Completo podem ser resolvidos em tempo polinomial, tornando-o um dos problemas abertos na computação e frequentemente debatidos também na matemática.

3.1.4 Classe NP-Difícil

O estudo de problemas de otimização NP-Difíceis em diferentes áreas da computação tem prosseguido em alta mesmo anos após a definição destes(SöRENSEN, 2015), seja para matemática aplicada, pesquisa operacional, meta-heurísticas ou inteligência artificial(BELLO et al., 2016).

Pode-se definir uma linguagem $L' \subseteq 0{,}1^*$ como NP-difícil se $L \leq_p L'$ para toda linguagem $L \in NP$, tal que L é NP-Completa se L' é NP-difícil e $L' \in NP$(OLIVEIRA, 2010).

Observando essa classe de problemas, é possível ver alguns consideravelmente conhecidos, dentre os quais estão PM, e o PCV, com diversas soluções de otimização propostas(GALLARDO; COTTA; FERNáNDEZ-LEIVA, 2008)(LAPORTE, 2006).

3.2 Problema da Mochila

É um problema clássico de otimização NP-Difícil que pode ser definido formalmente da seguinte forma: Se é necessário armazenar itens num espaço nomeado como mochila, onde o maior espaço possível deve ser preenchido sem que o peso máximo seja ultrapassado, de maneira que o lucro sob a coleta dos itens seja o máximo possível(CALDAS, 2004).

Como um problema clássico de otimização combinatória, é possível encontrar diferentes definições para PM, sendo que algumas podem permitir repetições, possibilitando que o mesmo item seja coletado várias vezes, e outras não. Há também situações em que embora possa haver repetição, os itens são limitados, não podendo haver uma adição infinita para estes(KELLERER; PFERSCHY; PISINGER, 2004).

PM é corriqueiramente associado à programação dinâmica, uma vez que suas resoluções correspondem com heurísticas e meta-heurísticas dessa categoria desde que definições como as de Dantzig(DANTZIG, 1957) e Glover(GLOVER, 1965) foram fundamentadas.

Embora PM enfoque soluções com programação dinâmica, pode-se observar que diversas outras heurísticas e meta-heurísticas podem ser aplicadas, tal como algumas resoluções mostradas por Gallardo, Cotta e Fernández-Leiva(2008).

3.2.1 Programação Dinâmica

De acordo com Ahrens(1992), programação dinâmica é um método para resolução de problemas computacionais, e busca aproximar-se de soluções ótimas por meio da decomposição de um problema complexo em sub-problemas menores, que sejam interdependentes e organizados em múltiplos estágios durante determinado horizonte de planejamento.

Segundo Dykstra(1984), um problema pode ser formulado com programação dinâmica se puder ser subdividido em estágios, de modo que as soluções de cada um dos estágios estejam interrelacionadas. Cada um dos estágios deve conter uma quantidade de estados possíveis, e o efeito sobre cada decisão é o de transformar o sistema de acordo com sua descrição do estado atual para outro estado, de modo que a condição necessária que deve ser satisfeita por um problema de programação dinâmica está implícita ao "Princípio da Otimalidade"de Belman(2010).

3.2.2 Variante binária (0-1)

Esta é uma importante variação de PM, comumente é o foco dos algoritmos que procuram solucionar este problema, uma vez que a dificuldade é reduzida de acordo com simplificações realizadas para uma situação específica.

A descrição original de PM possibilita ao ladrão colocar os itens de forma repartida em sua mochila, gerando assim uma quantidade muito maior de possibilidades de solução, demandando determinada sofisticação dos algoritmos que almejam resolver PM.

Esta variação consiste em limitar o ato de coleta de forma binária, então as únicas opções disponíveis são colocar o item como um todo na mochila ou não coloca-lo. Desta forma, a quantidade necessária de verificações sobre cada item é consideravelmente reduzida(WUIJTS, 2018).

Por conseguinte, esta variação possibilita testes em instâncias maiores do problema(TRUONG; LI; XU, 2013), uma vez que a redução do problema diminui a quantidade de computação necessária para alcançar a resposta aproximada.

3.3 Problema do Caixeiro Viajante

O PCV é um problema computacional NP-Difícil que pode ser aplicável a variadas situações(CHAN; MERCIER, 1989), e atualmente tem sido estudado e trago resultados por meio de diferentes técnicas, podendo ser dividido em vários outros sub-problemas, além de simplificações de acordo com o tipo de grafo utilizado, seja este simétrico ou assimétrico(PADBERG; GROTSCHEL, 1985).

O problema foi fundamentado com base nos caixeiros viajantes, profissão antiga na qual uma pessoa necessitava sair de uma cidade para realizar a venda de produtos em outras cidades, onde a menor rota de acesso entre estas cidades impacta diretamente no tempo de chegada, acabando por dar as primeiras pistas para o conhecimento dessa classe de problemas(LAPORTE, 2006).

Embora seja bastante reconhecido, o PCV é de autoria desconhecida, mas que pode ser observado com problemas sendo estudados inicialmente no século XIX, sendo mais difundido e definido com o nome que conhecemos somente no século posterior, sendo utilizado em diversas aplicações contemporâneas relacionadas ao movimento de pessoas, materiais e veículos no seu dia a dia(APPLEGATE et al., 2007).

O problema consiste na busca de um circuito hamiltoniano de menor distância pelos x nós de um grafo conexo de y nós, onde $y \geq x$(LAPORTE, 1992).

O modelo matemático do PCV nos concede fácil associação a problemas que envolvem logística, uma vez que sua formalização montou-se a partir de um. Sempre que

assuntos ligados à *Roteirização* de veículos são levantados, é comum que o PCV seja citado, tal que uma de suas principais aplicações da-se a redução de custos em casos de entregas(CUNHA, 2000).

Embora seu uso e associação seja comum a esta área, é fato que cada uma de suas aplicações necessita de um estudo a parte sobre o caso em que será feita a otimização, sendo essencial o uso de novas técnicas e considerações sobre novas variáveis no processo(MOURA; OLIVEIRA, 2004).

3.4 Problema do Ladrão Viajante

O Problema do Ladrão Viajante é um problema de otimização relativamente novo, introduzido por Bonyadi et al.(2013), sendo uma combinação de PM e do PCV. Por se tratar de uma junção, pode-se considerar que o PLV tem uma complexidade maior como um todo, uma vez que a resolução de um dos subproblemas afeta diretamente o outro.

De acordo com as definições e conceitos explorados anteriormente sobre o PCV e PM, se faz possível considerar PLV como um dos casos em que é necessário englobar um conjunto de questões e variáveis novas para resolução sobre determinado problema logístico.

3.4.1 Razão de formulação do problema

Como discutido na apresentação original do PLV Bonyadi et al.(2013) apresentam a formulação do novo problema, bem como cita os motivos que o levaram a elaboração do mesmo.

Problemas como o PCV e PM são considerados clássicos, já que puderam ser estudados profundamente por meio de diversos métodos. Muitos destes estudos buscam otimizar ou criar novas soluções, produzindo soluções que podem ser testadas contra instâncias e com parâmetros que fogem muito da realidade, onde muitas vezes a solução é baseada em uma instância do problema previamente e não no caso genérico, por este motivo Bonyadi et al.(2013) enxergam que algoritmos que surgem atualmente com base nessas instâncias possuem pouca aderência para aplicação na realidade.

Considerado o levantamento e observações realizados por Bonyadi et al, é proposto um novo problema que englobe partes destes dois problemas clássicos, de maneira que uma resolução moderna para o problema possa ser definida e que as soluções obtidas possuam uma boa aderência e aplicabilidade a problemas mais próximos da realidade.

3.4.2 Subproblemas

Como exposto, PLV pode ser dividido em subproblemas interdependentes, dado que se os subproblemas são resolvidos separadamente, mesmo que soluções ótimas sejam encontradas para cada, não garante que o problema principal seja resolvido também de forma ótima.

Nesta sessão trataremos os subproblemas do PLV, demonstrando o funcionamento interdependente de cada subproblema dentro do problema principal, além de uma melhor exploração sobre cada um dos detalhes da definição formal do problema.

3.4.2.1 Definição do Problema do Caixeiro Viajante para o Problema do Ladrão Viajante

Como exposto, o PLV possui subproblemas claramente definidos, sendo um destes o PCV, um clássico problema NP-Difícil de otimização. Neste problema existem n cidades, e as distâncias entre estas cidades são dadas pela matriz $D=\{d_{ij}\}$ (sendo está a distância entre as cidade i e j). Há um caixeiro viajante que precisa visitar todas as cidades, passando em uma a cada vez, de modo que o tempo gasto seja o menor possível. O caixeiro mantém uma velocidade constante (v_c)(BONYADI; MICHALEWICZ; BARONE, 2013), e a função objetivo deste problema pode ser dada pela seguinte fórmula:

$$f(\overline{x}) = \sum_{i=1}^{n-1}(t_{\mathrm{x_i,\ x_{i+1}}}) + t_{\mathrm{x_n,\ x_1}}, \overline{x} = (x_1, ..., x_\mathrm{n}) \tag{3.1}$$

Onde $\overline{x}$ representa o caminho percorrido pelo caixeiro, habitualmente chamado de *tour*, e contém todas as cidades exatamente uma vez, $t_{x_i,\ x_{i+1}}$ representa o tempo entre a cidade x_i e x_{i+1} e é calculado pela seguinte fórmula:

$$t_{\mathrm{x_i,\ x_{i+1}}} = \frac{d_{\mathrm{x_i,\ x_{i+1}}}}{v_\mathrm{c}} \tag{3.2}$$

3.4.2.2 Definição do Problema da Mochila Binária para o Problema do Ladrão Viajante

Outro dos subproblemas do PLV é o PM, um problema de otimização da classe NP-Difícil assim como PCV. Neste problema temos um ladrão que deseja encher sua mochila, considerando que existam m itens $I_1, ..., I_m$, sendo que cada um possui um valor p_i e um peso w_i. Não é possível que o ladrão leve todos os itens, dado que sua mochila possui um limite W.

O objetivo do ladrão consiste em maximizar seu lucro coletando a melhor combinação de itens para sua mochila. O problema pode ser modelado da seguinte forma:

$$maximizeg(\overline{y}) = \sum_{i=1}^{m} p_\mathrm{i} y_\mathrm{i}, \overline{y} = (y_\mathrm{i}, ..., y_\mathrm{m}) \tag{3.3}$$

$$subjectto \sum_{i=1}^{m} w_{\mathrm{i}} y_{\mathrm{i}} \leq W \tag{3.4}$$

Onde $\mathrm{y_i} \in \{0, 1\}$ e $\mathrm{y_i} = 1$ indica que o item i foi selecionado, *g(y)* calcula o valor total dos itens selecionados, e o objetivo final é maximizar g de modo que o peso total não exceda a capacidade da mochila (W).

3.4.3 Definição do Problema do Ladrão Viajante

Dado um número n de cidades com uma quantidade m de itens espalhados entre elas, e uma matriz $D{=}d_{ij}$ contendo as distâncias entre as cidades n_i e n_j, um ladrão deve visitar cada uma das n cidades, sendo uma por vez, e decidir quais dos itens da cidade deverá adicionado a sua mochila, e retornar para a primeira cidade ao fim do percurso.

O ladrão começa sua jornada na primeira cidade, e poderá selecionar itens desta cidade somente no começo de sua jornada. O ladrão não poderá selecionar itens desta cidade quando voltar para a mesma ao término do ciclo.

Quanto mais itens o ladrão coleta, mais pesada fica sua mochila e mais lenta fica sua velocidade. Qual o melhor caminho a percorrer e qual a melhor forma de coletar esses itens afim de obter o melhores benefícios?

Observando a descrição e métodos de resolução já definidos para o problema, pode-se notar a distinção do PLV em duas variantes, onde numa busca-se a maximização do lucro, ao passo que outra busca obter a minimização do tempo de *tour*(BONYADI; MICHALEWICZ; BARONE, 2013).

3.4.4 Modelos do problema

No artigo de formulação do problema, Bonyadi et al.(2013) propõem duas variantes para PLV, que foram denominadas como $\mathrm{TTP_1}$ e $\mathrm{TTP_2}$, onde TTP é abreviação do nome original *Travelling Thief Problem.*

$\mathrm{TTP_2}$ é uma variação de otimização bi objetiva, que busca minimizar o total de tempo percorrido bem como maximizar o lucro, com o adicional de que o valor dos produtos decai conforme o passar do tempo(WUIJTS, 2018).

3.4.4.1 Modelo $\mathrm{TTP_1}$

$\mathrm{TTP_1}$ é a variante mais estudada do problema até o momento (WUIJTS, 2018), dado que sua otimização é focada em um único objetivo, maximar o lucro obtido. É importante citar que a definição de $\mathrm{TTP_1}$ é um pouco diferente da regularmente utilizada na literatura, em razão da definição original permitir que o ladrão pegasse o mesmo item em cidades diferentes.

Não obstante, a literatura atual é focada em grande parte na variação criada por Polyakovskiy et al.(2014), na qual os itens são limitados a estarem presentes em somente uma cidade. Para tornar a interdependência dos problemas maior, novos parâmetros são introduzidos, sendo estes:

(a) A velocidade do ladrão é relacionada ao peso da mochila, sendo expressa pela seguinte fórmula:

$$v_{\mathrm{c}} = (v_{\max} - W_{\mathrm{c}} \frac{v_{\max} - v_{\min}}{W}) \tag{3.5}$$

Onde w_c é o peso atual da mochila, v_{max} e v_{min} são os valores de velocidade máxima e velocidade mínima do ladrão respectivamente, e W é o peso total que a mochila suporta. Vale ressaltar que esta velocidade é usada no cálculo do tempo entre as cidades pela expressão (3.1).

De acordo com essa fórmula, a velocidade do ladrão (v_c) diminui conforme ele possui uma maior quantidade de itens em sua mochila, posto isto, o ladrão viaja em v_{max} quando a mochila está vazia (W_c=*0*) e em v_{min} quando a mochila estiver cheia (W_c=W).

(b) A mochila utilizada pelo ladrão é alugada, de maneira que há um valor definido para usá-la. O valor utilizado é $\$R$ por unidade de tempo.

O objetivo do modelo de TTP_1 é maximizar a seguinte função:

$$G(x, z) = g(z) - R * f(x, z) \tag{3.6}$$

Onde g é o valor total dos itens recolhidos, R o valor de aluguel por unidade de tempo e f o tempo total levado para percorrer todas cidades. x e z são os resultados dos subproblemas, PCV e PM respectivamente. Note que a função para calcular a velocidade necessita de quais itens foram pegos e onde foram pegos, logo f precisa de x e z para poder calcular o tempo total.

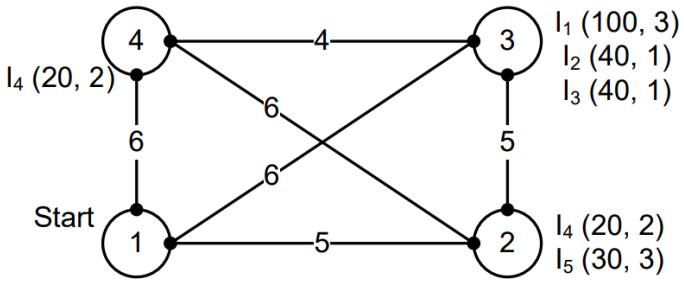

Figura 1 – Exemplo de uma instância para TTP_1. Fonte: The travelling thief problem: The first step in the transition from theoretical problems to realistic problems(BONYADI; MICHALEWICZ; BARONE, 2013).

Neste caso temos os produtos de i_1 até i_5 distribuídos entre 4 cidades, no qual cada produto possui seu valor e peso respectivamente. Há também a distância entre as cidades, formando um grafo não direcionado.

3.5 Busca Local

Um algoritmo de busca local, também referido na literatura como busca em vizinhança, é uma estratégia heurística utilizada na solução de problemas de otimização, onde o caminho para o objetivo é irrelevante, de maneira que a árvore de busca não é mantida(RODRIGUES, 2000).

É construído de forma a explorar o espaço de busca definido a partir de um estado inicial, que pode ser gerado por algum outro método ou aleatoriamente, permitindo melhora a cada iteração até que um estado ótimo seja atingido(OLIVEIRA; GUELPELI, 2013).

Métodos de busca local podem encontrar problemas em determinadas regiões do espaço de solução, conhecidos como ótimos locais, que se distanciam de uma solução ótima global para um determinado problema. As Meta-heurísticas são desenvolvidas para diminuir a probabilidade de se alcançar alguns destes ótimos locais, possibilitando convergirem para uma solução ótima global(OLIVEIRA; GUELPELI, 2013).

3.5.1 2-OPT

2-OPT é um algoritmo de otimização que utiliza de busca local, proposto por Croes(1958) para solucionar PLV, e consiste em obter uma rota que se cruza e reordena-la para tentar aprimorar a solução atual.

Uma pesquisa local completa com 2 opções compara todas as combinações válidas possíveis para a realização das trocas, assim, 2 arestas não adjacentes são eliminadas e reconectadas a outro par de arestas, onde há a verificação se a troca ocasionou melhora. Quando as arestas são reconectadas, um ciclo é formado, procedimento que é repetido para todos pares de arestas e, quando a iteração de comparação é terminada, o ciclo com menor custo é dado como a melhor troca, que é realizada em seguida. Este processo é iterado até que nenhum par de arestas sobre para checagem(CROES, 1958)(YAFRANI; AHIOD, 2017).

3.6 Algoritmos gulosos

Algoritmos relacionados a otimização normalmente utilizam de uma sequência de passos, onde cada um conta com certa quantidade de opções disponíveis. Algoritmos gulosos utilizam uma estratégia de otimização que visa sempre a escolha ótima destas

opções, tentando alcançar o ótimo global através destas escolhas, embora nem sempre seja possível(ROCHA; DORINI, 2004).

Comumente são escolhidos por terem implementação relativamente simples e serem efetivos, uma vez que suas escolhas são baseadas apenas na informação disponível, sem que os passos anteriores sejam guardados ou reconsiderados, o que torna desnecessário complexidades que envolvam desfazer escolhas anteriores(CORMEN et al., 2012)(BRASSARD; BRATLEY, 1995).

3.7 Computação Evolucionária

A Computação Evolucionária(CE, ou Computação Evolutiva) constitui uma família de algoritmos com enfoque em abordar problemas complexos através de modelos computacionais de processos evolutivos, onde um conjunto inicial de soluções é gerado e atualizado de maneira iterativa, fazendo com que uma população de soluções passe por uma seleção natural ou artificial, de modo a evoluir e adaptar as mais adequadas(OLAZAR, 2007).

Estas técnicas evolutivas baseiam-se diretamente na imitação da natureza a partir do Neo-Darwinismo(FOGEL; OWENS; WALSH, 1966), tanto nos processos de reprodução quanto na mutação e seleção dos indivíduos para gerações futuras. A tentativa de reproduzir estes princípios estaticamente através de algoritmos parte da ideia de que a natureza otimiza seus mecanismos para resolver um ou mais problemas de forma natural alcançando uma boa solução(POLI; LANGDON; MCPHEE, 2008).

3.7.1 Algoritmos Evolucionários

Chama-se Algoritmos Evolucionários(AEs, ou Algoritmos Evolutivos) a implementação dos processos de CE por meio de buscas estocásticas que evoluem um conjunto de estruturas e selecionam de modo iterativo as mais adequadas. Segundo Olazar(2007), AEs utilizam de um vocabulário derivado da genética natural de acordo com os seguintes pontos:

1. **População:** É um conjunto de indivíduos, ou seja, um repertório de soluções candidatas que interagem entre si e competem pela permanência na população a cada iteração.

2. **Indivíduos:** Conhecidos também como cromossomos, cada indivíduo é uma solução que controla a herança de um ou vários caracteres. Cada indivíduo é codificado em um conjunto de símbolos chamado genes, que contém diferentes valores chamados de *alelo*(GABRIEL; DELBEM, 2008). A posição do gene em um cromossomo é denominada *locus*.

3. **Operadores genéticos:** São os métodos que manipulam a informação genética de cada indivíduo da população afim de produzir novos indivíduos que são variações da população original. Podemos destacar os seguintes operadores genéticos comumente utilizados em AEs:

   a) **Mutação:** É a alteração de um ou mais alelos da genética do indivíduo. A cada iteração mudanças randômicas são aplicadas, buscando diferentes regiões do espaço de soluções e evitando ótimos locais.

   b) **Cruzamento:** Do termo em inglês *crossover*, é o processo em que as características de dois cromossomos(chamados de pais) se misturam e geram novas cadeias cromossômicas dos descendentes(chamados de filhos), visando sempre preservar bons cromossomos que levaram às melhores soluções do problema.

   O cruzamento conta com a etapa de recombinação, que divide os genes de cada indivíduo e troca-os de posição entre si, explorando as zonas mais propícias no espaço de busca. Podemos verificar um exemplo disso na figura (2).

Pai 1	1	1	0	0	1	1	1
Pai 2	1	1	1	0	0	1	0
Filho 1	1	1	1	0	1	1	1

Figura 2 – Exemplo de *crossover* com um ponto de corte. Fonte: Utilização de algoritmos genéticos para otimização de soluções para o timetabling escolar.(SIMãO, 2013).

   De acordo com Simão(2013), os seguintes operadores de cruzamento são comumente utilizados na literatura:

   i. **Cruzamento com corte em 1 ponto (1Px):** Assim como demonstrado na figura (2), a partir de um corte px, a parte direita dos genes é herdada do $Pai1$, enquanto a parte esquerda é herdada do $Pai2$;

   ii. **Cruzamento com corte em 2 pontos (2Px):** É uma técnica semelhante a $1Px$, porém usa dois pontos de corte, herdando duas partes de um pai e a parte restante de outro;

   iii. **Cruzamento com múltiplos pontos (MPx):** É uma generalização de $1Px$ e $2Px$, onde múltiplos cortes px são realizados;

   iv. **Cruzamento segmentado (Sx):** Semelhante ao MPX, utiliza de um número de cortes variável, mas o número n de pontos de corte é alterado todas as vezes que é executado.

4. **Função de aptidão (*fitness*):** É o valor que indica o quanto cada indivíduo é adequado perante a população. Em problemas de otimização, o *fitness* pode ser o

custo da solução(GABRIEL; DELBEM, 2008). A determinação deste valor chama-se avaliação da função de aptidão.

5. **Seleção:** É o procedimento que realiza a escolha de indivíduos que passarão à próxima geração com base nos resultados obtidos pela função de aptidão.

De acordo com Trindade e Ochi(2006), um AE pode ser descrito da seguinte forma:

Algoritmo 1 Evolutivo

1: Gera população inicial(Pop)
2: nº de gerações = 1
3: melhor indivíduo = melhor indivíduo(Pop)
4: **enquanto** (nº de gerações < nº máximo de gerações) **faça**
5: $Pop_{+1} \leftarrow \emptyset$
6: Reprodução(Pop, Pop_{+1})
7: $Pop \leftarrow Pop_{+1}$
8: BuscalLocal(Pop)
9: Atualizar melhor indivíduo
10: nº de gerações++
11: **fim enquanto**
12: **imprimir** melhor indivíduo

3.7.1.1 Algoritmos Meméticos

Um algoritmo memético(AM) é uma extensão de um algoritmo genético tradicional, e utiliza de busca local para reduzir a probabilidade de convergência prematura. É uma técnica de otimização baseada na combinação sinérgica de ideias tiradas de outras meta-heurísticas, como busca populacional de AEs e busca local(MOSCATO; COTTA, 2003).

A ideia principal que levou ao surgimento dos AMs baseava-se em combinar conceitos e estratégias de diferentes meta-heurísticas para tentar aproveitar suas principais vantagens(MOSCATO; COTTA, 2003). O termo "memético"é um análogo do gene no contexto de evolução natural, e vem do termo "meme", criado em 1976 por Richard Dawkins(DAVIS, 1991). É como uma unidade informativa que se multiplica de cérebro em cérebro ou entre locais onde a informação é armazenada. Os 'memes' espalham-se na memória partindo de um cérebro para outro em um processo que, em um sentido amplo, pode ser chamado imitação.

Desta forma, um AM trabalha com as melhorias individuais das soluções em cada um dos agentes junto com processos cooperativos e competições do tipo populacional(MOSCATO; COTTA, 2003). Podendo ser visto como uma estratégia de busca local onde os agentes de otimização cooperam e competem, quando utilizado em problemas de otimização um AM contém certa flexibilidade, uma vez que sua aplicação pode utilizar de heurísticas pré-existentes(NORMAN; MOSCATO, 1989).

3.8 Estado da arte do Problema do Ladrão Viajante

Após a formalização do problema por Bonyadi et al.(2013), alguns estudos sobre o problema foram realizados, definindo soluções de otimização com heurísticas de solução única e evolucionárias(WUIJTS, 2018).

3.8.1 Heurísticas de solução única

DH(Heurísticas baseadas em densidade, traduzido de *Density-based Heuristics*) e $CoSolver$(Co-solucionadores) foram heurísticas propostas por Bonyadi et al.(2013) no estudo de formulação do problema. DH utiliza um *tour* gerado pela heurística de Lin-Kernighan(APPLEGATE; COOK; ROHE, 2003), e busca ordenar a mochila de acordo com esse *tour*, utilizando um esquema de pontuação considerando o impacto do aluguel da mochila durante a coleta. $CoSolver$ busca decompor PLV em dois subproblemas conforme demonstrado na seção 3.4.2.

Em testes com instâncias pequenas com um número de itens $n < 25$, $CoSolver$ apresentou resultados melhores para a maioria dos casos, onde é confirmado que considerar a interdependência dos problemas traz um impacto positivo na solução(BONYADI; MICHALEWICZ; BARONE, 2013).

Afim de obter melhores testes de *benchmarks* para PLV, Polyakovskiy et al. (2014) propuseram os algoritmos *Simple Heuristic*(SH), *Random Local Search*(RLS) e (1+1) *Evolutionary Algorithm*($(1+1)-EA$). Os três algoritmos são gulosos e utilizam da mesma solução inicial $S0$ produzida pela heurística de Lin-Kernighan para realizar a otimização da solução. SH é relativamente parecido com o proposto por DH, com a diferença que DH calcula o aumento real de um item a ser coletado, enquanto SH se aproxima disso calculando o ganho total da adição de um item em um *tour* vazio.

El Yafrani e Ahiod(2016a) propuseram os algoritmos de HC JNB e $J2B$, onde JNB usa uma combinação de $BitFlip$ e trocas entre cidades consecutivas, já $J2B$ parte da combinação de $BitFlip$ com o algoritmo $2-OPT$. Por utilizarem de uma vizinhança combinada $O(n*m)$ e $O(n^2*m)$, os algoritmos sofrem de um grande tempo de execução mesmo para as instâncias $n < 25$ itens. Para viabilidade da execução, uma implementação da triangulação de Delaunay foi feita assim como já utilizado por Mei, Li e Yao(2014), de modo a restringir a vizinhança do candidato avaliado para a troca. Com os estudos de JNB e $J2B$ concluiu-se que uma boa solução inicial oferece resultados melhores que soluções iniciais aleatórias na maioria das vezes. JNB e $J2B$ ofereceram resultados melhores que $(1+1)-EA$ e RLS.

Faulkner et al.(2015) propõem uma série de algoritmos que utilizam de heurísticas $PackingIterative$, que utilizam de sub-rotinas de $Pack$, uma heurística gulosa definida por:

$$Pontuação_2(I_{\mathrm{ik}}, \alpha) = \frac{{p_{\mathrm{ik}}}^{\alpha}}{{w_{\mathrm{ik}}}^{\alpha} \times d_{\mathrm{i}}} \tag{3.7}$$

onde p_{ik} e w_{ik} são o lucro e peso de determinado item I_{ik}, e d_{i} é o restante do *tour* em uma cidade i. Como o expoente α influencia a pontuação, o sucesso da heurística depende de um bom valor para α, que altera diretamente a coleta.

Uma vez que $PackingIterative$ obtém soluções diferentes de acordo com o α, o algoritmo toma um α inicial e, através de comparações, diminui ou aumenta seu valor para obter uma boa solução. Faulkner et al.(2015) propuseram 11 variações para o mesmo algoritmo, nomeados $S1 - S5$ e $C1 - C6$, que utilizam de múltiplas buscas locais para cada vez que o algoritmo é reiniciado, incluindo $BitFlip$, $Insertion$ e $(1+1) - EA$. Todos iniciam com uma solução gerada por Lin-Kernighan e a variante $S5$ apresentou os melhores resultados.

Proposto por El Yafrani e Ahiod(2017) em um novo estudo, $CS2SA$ é um algoritmo baseado em $CoSolver$, que a partir de uma solução inicial tenta otimizar o tour com $2-OPT$ reduzido pela triangulação de Delaunay(MEI; LI; YAO, 2014), para então otimizar o plano de coleta utilizando *Simulated Annealing*. O processo é repetido por diversas vezes até que não tenham mais melhoras a serem feitas.

$CS2SA$ e $S5$ apresentaram as soluções próximas, onde $CS2SA$ mostrou conseguir resultados melhores para instâncias em que a mochila tinha uma maior capacidade(YAFRANI; AHIOD, 2017).

3.8.2 Aproximação Exata

Wu et al.(2017) propôs abordagens exatas para PLV utilizando técnicas de programação dinâmica, programação por restrições, que especifica restrições e critérios que uma solução deve cumprir, e *branch and bound*, que busca desenvolver uma enumeração inteligente das soluções candidatas à solução ótima inteira de um problema.

Devido a alta dificuldade do problema, a complexidade de tempo das soluções para instâncias grandes é alta demais, crescendo exponencialmente com o número de cidades. Em amostras pequenas o encontro da solução ótima auxiliou o aprimoramento e comprovou a eficácia de algoritmos propostos, tal como MA2B(YAFRANI; AHIOD, 2016b)(WUIJTS, 2018).

3.9 Hill Climbing

Hill Climbing é uma busca heurística usada para problemas de otimização matemática que utiliza de busca local. Garante uma boa solução em tempo razoável, partindo de uma solução inicial e realizando aprimorações ao longo das iterações(RAWAT, 2018).

Se trata de um algoritmo de simples aplicação, com resultados relativamente bons e acaba por ser uma escolha frequente para resolução de uma alta gama de problemas, especialmente para inteligência artificial, embora possa não entregar as melhores soluções de acordo com a situação em que está inserido, uma vez que é um algoritmo guloso(VIEIRA; MARQUES; CASTRO, 2015).

Como algoritmo guloso, HC escolhe a melhor alternativa atual de maneira iterativa, gerando o resultado a partir do caminho com as melhores opções de cada laço. De acordo com Zuben e Castro(201-?), o algoritmo de HC pode ser descrito da seguinte maneira:

Algoritmo 2 Hill Climbing

1: $S \leftarrow$ solução inicial
2: **enquanto** (S não for ideal ou o tempo não se esgote) **faça**
3: $R \leftarrow$ NovaSolução(S)
4: **se** (Qualidade(R) > Qualidade(S)) **então**
5: $S \leftarrow R$
6: **fim se**
7: **fim enquanto**
8: **retorna** S

De acordo com a definição, poderíamos determinar facilmente versões para PM e PCV, uma vez que seria fácil encontrar uma solução que atendesse ao problema, tentando preencher a mochila com itens quaisquer ou determinando um caminho aleatório até que todos os nós buscados fossem visitados, mas que possivelmente estaria longe do ideal de otimização. Para aperfeiçoar estas soluções encontradas, bastaria iterar com a técnica de HC para que a solução melhorasse em comparação com a atual.

3.9.1 Problemas e Limitações

Como qualquer algoritmo, HC acaba por apresentar algumas situações em que o desempenho do algoritmo pode ser prejudicado, por conta dos dados e solução do cenário em que está sendo aplicado. Essas situações podem ser resumidas em três problemas locais, e são definidos como máximos locais, planícies, e as encostas e picos(RUSSELL; NORVIG, 2016).

Caso o problema conte com máximos locais, o algoritmo pode acreditar estar na melhor solução quando está apenas numa boa solução de determinada região das soluções. Pode-se ter uma melhor noção dos máximos locais e do máximo global observando o gráfico apresentado na figura (3).

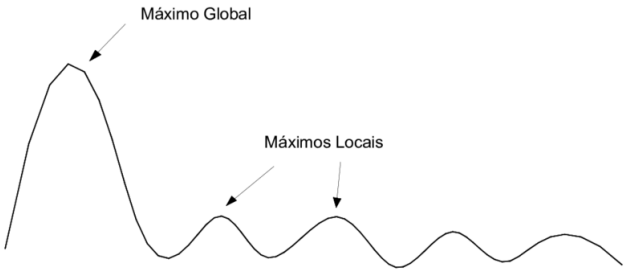

Figura 3 – Demonstrativo de máximo global e máximos locais. Fonte: Algoritmos evolutivos aplicados a síntese e otimização de sistemas de ancoragem(ALBRECHT et al., 2019)

Para planícies podemos considerar uma determinada área dos dados ou soluções que são extremamente próximos, a ponto de ocasionar dificuldades para o algoritmo assumir um lado para avançar, podendo correr o risco de manter-se num ponto não ideal.

Em um pico, o valor acaba por subir pouco caso algoritmo se mova em uma direção, mas acaba por cair em excesso se o movimento for em direção contrária.

3.9.2 Variantes

Como uma técnica que pode obter soluções a uma grande variedade de problemas, podemos facilmente encontrar variantes de HC, que melhoram um caso específico atendido pelo algoritmo ou apresentam melhorias consideráveis ou mesmo aleatórias para a técnica como um todo.

3.9.2.1 Steepest Ascent

Podemos considerar a *steepest ascent* como um tipo específico de HC, dado que HC simples busca maximizar uma função em função dos vizinhos investigando o melhor custo para continuar a subida, enquanto em *steepest ascent* minimiza-se a função observando a inclinação da vizinhança local se dirigindo para a direção mais íngreme. Sendo assim, considera-se que em *steepest ascent* assume-se que você pode otimizar uma função contínua e calcular seu gradiente em determinado estado(WRIGHT, 2015).

Com *steepest ascent* obtem-se resultados relativamente mais precisos que na versão simples do algoritmo, porém, a quantidade de problemas que se pode atacar é considerada relativamente menor.

O algoritmo é descrito por Russell(RUSSELL; NORVIG, 2016) da seguinte maneira:

Algoritmo 3 Hill Climbing *Steepest Ascent*

Entrada: *problema*

1: $atual \leftarrow$ CriarNó(EstadoInicial[$problema$])
2: **enquanto** *Solução* não for ideal ou o tempo não se esgote **faça**
3: $vizinho \leftarrow SucessorDeMaiorValor[atual]$
4: **se** $(Custo(vizinho) \leq Custo(atual))$ **então**
5: $atual \leftarrow vizinho$
6: **fim se**
7: **fim enquanto**
8: **retorna** Estado[$atual$]

3.9.2.2 Estocástico

O HC estocástico foi projetado como uma técnica de busca local para ser utilizada em domínios discretos com vizinhos explícitos, como a otimização combinatória, e é capaz de ser usado para refinar uma solução após a execução de um algoritmo qualquer de busca global.

A versão estocástica busca iterar o processo de selecionar aleatoriamente um vizinho para uma solução candidata e apenas aceitá-lo se resultar em um melhor ou igual custo. O algoritmo escolhe aleatoriamente entre os vizinhos para realizar os movimentos acima, fazendo com que a probabilidade de seleção possa variar com a inclinação do movimento.

Por se tratar de um algoritmo estocástico, pode ser reiniciado várias vezes para que alcance um resultado aprimorado. O uso desta reinicialização é conhecida como *Multiple Restart Hill Climbing*, permitindo também execuções paralelas conhecidas como *Parallel Hill Climbing*, que pode conter diversos relacionamentos com o intervalo especificado(BROWNLEE, 2011).

Algoritmo 4 Hill Climbing Estocástico

Entrada: $ÍTER_{máx}$, Tamanho do problema

1: **para** $ÍTER_i$ até $ÍTER_{máx}$ **faça**
2: candidato $\leftarrow$ VizinhoRandômico(atual)
3: **se** Custo(candidato) $\leq$ Custo(atual) **então**
4: atual $\leftarrow$ candidato
5: **fim se**
6: **fim para**
7: **retorna** *atual*

3.9.2.3 Reinicialização Aleatória

A reinicialização aleatória é um meta-algoritmo que trabalha iterativamente utilizando HC Simples a partir de uma variável aleatória inicial x_0. Após a primeira solução ser

encontrada, o algoritmo roda mais n vezes, substituindo a solução atual por uma solução melhor ou igual até que não existam variações significativas entre cada uma das n soluções obtidas.

A grande vantagem da reinicialização aleatória da-se por não serem necessários grandes cuidados para se obter uma condição inicial, partindo de um ponto qualquer.(O'NEIL; BURTSCHER, 2015)

3.10 Simulated Annealing

O SA é uma meta-heurística proposta por Kirkpatrick et al.(1983), que embora conte métodos probabilísticos, utiliza busca local para obter uma solução assim como HC. O algoritmo se baseou em processos da metalurgia envolvendo termodinâmica para fundamentar-se, tendo esta como motivo para a origem da metáfora do uso de *annealing*[1] em seu nome(LAARHOVEN; AARTS, 1987).

O algoritmo de SA herda propriedades de acordo com as analogias de termodinâmica para otimização, tendo fundamentações teóricas baseadas em cadeias de Markov. É considerado um poderoso algoritmo, dada a possibilidade de demonstrar matematicamente a convergência assintótica para o conjunto de soluções ótimas globais(AARTS; KORST, 1989).

Embora a busca por um ótimo global acentue a qualidade da solução entregue pelo algoritmo, para problemas de otimização combinatória de grande porte a resolução pode não ser obtida em tempo computacional admissível, posto que o comprimento da cadeia de Markov seria tão extensa a ponto de levar décadas ou centenas de anos para encontrar o ótimo global.

Para iniciar, SA necessita de uma temperatura inicial T alta, que será reduzida com decorrer da execução, o número de iterações ou um limite de tempo de processamento, além do fator de redução de temperatura, que reduzirá a probabilidade de aceitação de soluções com degradação da função objetiva(AARTS; KORST, 1989).

De acordo com Downsland(1993), SA pode ser representado de acordo com o algoritmo (5).

Cada uma das entradas é uma variável selecionada, onde s_0 é uma solução inicial, t_0 é a temperatura inicial maior que zero, e α é o fator de redução de temperatura, que pode adotar diferentes estratégias(DOWNSLAND, 1993).

Levando em consideração as características de SA, vemos que sua fácil implementação computacional é um dos pontos mais atrativos do algoritmo, sendo que a quantidade de parâmetros necessários para seu funcionamento é relativamente baixa.

[1] Recozimento

Algoritmo 5 Simulated Annealing

Entrada: s_0, t_0, α

```
enquanto (condição de parada = verdade) faça
    enquanto (não chegar ao número de repetições) faça
        aleatoriamente selecione s ∈ N(s_0)
        δ = f(s) − f(s_0)
        se (δ < 0) então
            s_0 ← s
        senão
            Gere um número aleatório entre (0, 1)
            se (x < exp(−δ/t)) então
                s_0 ← s
            fim se
        fim se
    fim enquanto
    t ← α(t)
fim enquanto
```

3.10.1 Problemas e Limitações

Embora SA seja de fácil implementação e parta em busca de um máximo global com alta assertividade, demanda da utilização de algum mecanismo de controle para soluções repetidas, uma vez que segundo sua base teórica, utiliza-se de um sistema sem memória para prova de convergência. Ou seja, necessita-se de algum sistema de filtros ou listas para controle dessas soluções, embora a convergência assintótica não funcione adequadamente para termos práticos de otimização combinatória.

Embora o mecanismo de soluções de piora com uma probabilidade decrescente seja consideravelmente poderoso e mostre locais com ótimas soluções, se faz necessário a incorporação de outros algoritmos de busca local para melhora de performance, uma vez que o encontro de um ótimo local possa ser otimizado a partir de uma nova busca.

3.11 CoSolvers para a interdependência do PLV

Conforme proposto por Bonyadi et al.(2013), se faz necessário um *CoSolver* para resolução de PLV, que é um solucionador que busca desagregar o problema geral em uma função que considera a interdependência de cada um dos dois componentes.

Com base na solução inicial, PLV é decomposto para PCV com PM, onde o *tour* x' de melhor custo encontrado é combinado com a última coleta z' e otimizado em PLV em objetivo de uma função G, uma vez que a função de lucro total g não depende do *tour* realizado. Sendo assim, pode-se considerar minimizar o custo total de viagem T ao invés de maximizar G, tal que $T(x, z) = R \times f(x, z)$.

Analisa-se PM no caminho para obter o melhor plano de coleta z', tal que o ganho total G seja maximizado com o *tour* x' descrito anteriormente(YAFRANI; AHIOD, 2017).

Resumidamente, as duas fases são solucionadas separadamente considerando a interdependência de cada um destes subproblemas, possibilitando a união das soluções obtidas com a viagem T minimizada e ganho G maximizado.

3.12 MA2B

MA2B é um AM que usa técnicas de busca local 2-OPT e bit-flip com um *crossover* baseado em MPX, com base em uma população de 40 indivíduos e uma taxa de seleção de 75% (YAFRANI; AHIOD, 2016b).

O algoritmo baseia-se em 6 passos, sendo eles:

1. Busca Local Restritiva: É utilizada como uma técnica de intensificação, onde primeiramente há aplicação de uma pesquisa 2-OPT para melhora do *tour* seguida de uma busca por bit-flip para melhora do plano de coleta.

   Considerando o custo que estas buscas locais tem enquanto o AM é executado, limitações são aplicadas para reduzir o tempo de execução, sendo tipicamente adotado um limite de 50 iterações na inicialização e 10 durante o processo de pesquisa.

2. Estratégia de Inicialização: MA2B necessita de um *tour* inicial, fazendo com que este se torne um ponto de impacto para o desempenho do algoritmo, uma vez que a heurística de pesquisa depende diretamente disso, principalmente quando considerando os primeiros *tours*.

   Dada a importância de heurísticas para o *tour* inicial, desde a formulação do PLV muitas soluções foram elaboradas, sendo subsequentemente sofisticadas para obter um melhor resultado. O que temos em grande parte dos PLVs de última geração é a heurística de *Chained Lin Kernighan*(MEI; LI; YAO, 2014), que já se mostrou eficaz para uma gama de algoritmos solucionadores do PLV(YAFRANI; AHIOD, 2016a)(WUIJTS, 2018).

   Utilizando os fatores acima, podemos seguir o seguinte processo para cada solução na população:

   a) A partir de um passeio aleatório, utilize a heurística de *Chained Lin Kernighan* com um número de iterações limitado de acordo com o tamanho da instância utilizada no problema.

   b) Utilize uma heurística gulosa para gerar um *picking plan*.

   c) Aplique a busca local restritiva

A partir desta aproximação é possível coletar boas soluções em um bom tempo para aplicação da seleção(YAFRANI; AHIOD, 2016a).

3. Seleção: Utilizando uma população de torneio de 6 indivíduos, há uma seleção para gerar 2 soluções pai.

4. *Crossover*: Para o *crossover* é utilizado a técnica de *Maximal Preservative Crossover*(MICHALEWICZ; FOGEL, 2013), com um processo que pode ser dividido em duas etapas:

   a) A partir de um MPX aplicado aos *tours* dos pais, um *tour* descendente é gerado.

   b) Para gerar o plano de coleta descendente, o *tour* anteriormente obtido é empregue para determinar o estado de cada um dos itens, dessa forma caso a cidade que contenha um item seja herdada do primeiro pai, o estado do item também é herdado do primeiro pai, se não o estado é herdado do segundo pai.

   Embora outros operadores de *crossover* possam ser empregues, é mostrado por Dzubera e Whitley(1994) que algoritmos mais separativos como MPX podem apresentar bons resultados em AMs. A eficácia de MPX na estrutura de MA2B também foi comprovada por Yafrani e Ahiod(2016b), onde testes com MPX mostraram resultados mais eficazes que ERX(*Edge Recombination Crossover*)(WHITLEY; STARKWEATHER; SHANER, 1991), PX(*Partition Crossover*)(WHITLEY; HAINS; HOWE, 2009) e OX(*Order Crossover*)(DEEP; MEBRAHTU, 2011).

5. Mutações: As mutações são geradas sempre que herdeiros já existirem na população. A mutação é dividida em nas seguintes etapas:

   a) Aplique um movimento de *double-bridge* duas vezes no *tour*, fazendo com que o algoritmo avance para o dobro de áreas de cidades(LIN; SUN; SALOUS, 2016).

   b) Gere um novo plano de coleta a partir de uma heurística gananciosa.

   c) Aplique a busca local restritiva.

6. Critério de Parada: A busca termina apenas quando um dos três critérios é atingido:

   a) Número máximo de iterações;

   b) O limite de gerações sem melhoria;

   c) Tempo limite de execução.

3.13 Algoritmos baseados em *CoSolvers*

Proposto por Bonyadi et al.(2014), *CoSolver* é um algoritmo abstrato para resolver PLV. Com base na ideia de decomposição do problema, propostas surgiram posteriormente utilizando diferentes algoritmos para lidar com cada componente do problema.

3.13.1 CS2SA

Criado para a resolução do PLV por Yafrani e Ahiod(2016a), CS2SA é uma heurística de solução única baseada numa combinação de *Simulated Annealing* e 2-OPT. Baseado no Framework de *CoSolver*, a partir de um *tour* de entrada inicial o algoritmo utiliza de 2-OPT para melhorar o *tour* e a heurística de *Simulated Annealing* para melhorar o plano de coleta.

Para PCV há melhorias a partir de uma pesquisa de vizinhança, onde a triangulação de Delaunay(MEI; LI; YAO, 2014) seleciona os melhores candidatos no espaço de busca e 2-OPT realiza a seleção dos melhores candidatos.

Com os candidatos selecionados, a meta-heurística *Simulated Annealing* é aplicada para o subproblema de PM, gerando vizinhos aleatórios e utilizando um operador *bit-flip*. A partir desse ponto, há um loop interno que é usado para verificar mais possibilidades para uma temperatura fixa.

Sempre que houver aprimoramento, os passos de 2-OPT e *Simulated Annealing* são executados novamente, até que não tenha mais aprimoramento ou o tempo limite definido para o algoritmo seja alcançado.

3.13.2 CS2B

CS2B(*CoSolver-based with 2-OPT and Bit-flip*) é uma solução baseada em algoritmos de busca local proposta por Yafrani e Ahiod(2016a), que itera entre os solucionadores de PM e PLV a partir de um *tour* inicial.

A partir de um *tour* inicial, uma heurística de inserção e eliminação é utilizada para iniciar o plano de coleta, que é enviado junto do *tour* para um otimizador do PLV que usa a heurística de 2-OPT. A partir deste otimizador, o *tour* resultante com o plano de coleta é enviado a um otimizador de PM que aplica *bit-flip* para a melhora da seleção de itens. Este processo é repetido até que não haja melhoria ou alcançar um limite de tempo.

4 Desenvolvimento

Neste capítulo busca-se explanar a elaboração de uma proposta com base nos estudos da revisão bibliográfica realizada, na qual informações sobre o PLV e as heurísticas que compõem MA2B, CS2SA e CS2B foram levantadas tal como informações acerca dos próprios algoritmos, além de pontos relevantes sobre os subproblemas e variantes.

É pretendido implementar as técnicas propostas por Yafrani e Ahiod(2016a)(2016b) para resolução de PLV, e a partir destas realizar uma execução sobre determinado número de instâncias e elaborar um quadro de resultados que permita uma comparação direta sobre as melhores soluções para cada instância.

Buscamos verificar o tempo de execução, os valores de lucro e o desvio padrão com base nas múltiplas execuções sobre cada uma das instâncias, e se possível correlacionar com valores obtidos por outras pesquisas utilizando algoritmos que também passaram por comparações pelos alvos deste trabalho.

4.1 Configuração Experimental

Realizaremos testes com base na biblioteca de *benchmark* de Polyakovskiy et al.(2014), que também considera os parâmetros:

- **Problemas base para PLV:** as 81 instâncias disponibilizadas na *TSPLIB*;
- **Tipos de PM (T):** não correlacionado(ou *uncorrelated*)(1), não correlacionado com pesos semelhantes(ou *uncorrelated with nsimilar weights*)(2), fortemente correlacionados(ou *bounded strongly correlated*)(3), onde T representa o tipo;
- **Categoria de capacidade da mochila (C):** 1,...,10, de tal forma que quanto maior a capacidade da mochila, maior o valor de C;
- **Fator do item (F):** 1, 3, 5, 10, tal que F representa o número de itens por cidade

Consideraremos os seguintes problemas/instâncias: *eil76, kroA100, ch130, u159, a280, u574, u724.*

As instâncias serão executadas de modo diversificado tal como apresentado por Yafrani e Ahiod(2016b), inspiradas nas categorias da CEC 2014(*Competition on Optimization of Problems with Multiple Interdependent Components*):

- **Categoria 1:** (T=3, C=1, F=1);

- **Categoria 2:** (T=2, C=5, F=5);
- **Categoria 3:** (T=1, C=10, F=10);

CS2SA, CS2B e MA2B serão implementados em Java com um tempo limite de execução padronizado por 10 minutos(600 segundos).

4.2 Solução Inicial

Com base no levantamento realizado, a solução inicial selecionada é um ponto fundamental para se obter o melhor desempenho de HC e SA(desde que HC não utilize de reinicialização aleatória), sendo necessário os devidos cuidados para a adoção de uma estratégia.

Os algoritmos de Lin-Kernighan, Boruvka e 2-opt são frequentemente utilizados como heurísticas para obter um *tour* inicial para PCV, porém, casos como PLV devem considerar também uma solução inicial que seja minimamente razoável para a coleta dos itens.

Levando em consideração as questões de interdependência, utilizaremos a implementação de Concorde para a heurística de Lin-Kernighan, escrita por Applegate, Cook e Rohe(2003), capaz de ser utilizada para grandes instâncias do PLV, para obter o *tour* S_0, em complemento com a heurística proposta por Mei et al.(2014) para preencher a mochila com itens.

A técnica para preenchimento consiste na busca de três aproximações para se conseguir o melhor custo para a mochila. Com a busca realizada, se é feito um *bit-flipping* para descarte de qualquer item de valor ruim para a coleta, onde se realiza uma pesquisa simples de troca nos itens inseridos para que o descarte seja feito utilizando a função objetivo como base.

4.3 Algoritmos Base

Para a comparação objetiva deste trabalho, se faz necessário realizar a construção de três algoritmos base para resolução dos *CoSolvers* e construção das técnicas referenciadas em (3.12) e (3.13).

4.3.1 *2-OPT steepest ascent hill climbing*

Para resolução de PCV com PM, uma das heurísticas utilizadas para resolução e análise dos resultados será a descrita por El Yafrani e Ahiod(2017), que utiliza de *2-OPT steepest ascent hill climbing*, onde uma vizinhança N é gerada a cada iteração, utilizando

uma técnica de recuperação de valor objetivo para avaliar os vizinhos gerados. Sempre que um valor objetivo melhor é encontrado, seus índices e tempo são diretamente guardados. Essas iterações são continuadas e permanecem sempre atualizando a melhor solução até que não seja possível encontrar melhoria. A aplicação de 2-OPT para CS2SA, CS2B e MA2B é mostrada no algoritmo (6).

Considerando a aplicação de 2-OPT para o primeiro *CoSolver* nos três algoritmos, é certo dizer que ambos os algoritmos terão um impacto inicial muito parecido na execução do primeiro *CoSolver*.

Algoritmo 6 Heurística de busca 2-OPT para PLV com PM

1: $s \leftarrow$ solução inicial
2: $f \leftarrow$ tempo de viagem inicial
3: $T \leftarrow$ inicialize o parâmetro de limite
4: **enquanto** (não melhorar) **faça**
5: $\quad melhorado \leftarrow false$
6: $\quad$ **para** ($\overline{s}$ em $N(s)$) **faça**
7: $\quad\quad \overline{f} \leftarrow$ avalie $\overline{s}$ usando a técnica de recuperação de valor objetivo
8: $\quad\quad$ **se** ($\overline{f} - f < T$) **então**
9: $\quad\quad\quad i_{melhor} = i$
10: $\quad\quad\quad j_{melhor} = j$
11: $\quad\quad\quad f = \overline{f}$
12: $\quad\quad\quad melhorado = true$
13: $\quad\quad$ **fim se**
14: $\quad$ **fim para**
15: $\quad$ **se** ($melhorado$) **então**
16: $\quad\quad s \leftarrow$ aplique a troca 2-OPT em s a i_{melhor} e j_{melhor}
17: $\quad\quad$ recupere os vetores do histórico
18: $\quad$ **fim se**
19: **fim enquanto**

4.3.2 *Bit-flip*

O *CoSolver* para PM com *tours* pode ser resolvido utilizando *bit-flip*, que consiste no preenchimento da mochila a partir da realização de trocas e comparação dos itens utilizando a função objetivo como base.

A aplicação de *bit-flip* será utilizada diretamente para MA2B e CS2B, sendo a meta-heurística fundamental de coleta.

4.3.3 Simulated Annealing

Para lidar com o segundo *CoSolver* em CS2SA, utilizamos a técnica de SA adaptada vista no algoritmo (7), onde um candidato aleatório é verificado em uma iteração interno de modo a explorar uma maior quantidade de possibilidades.

Uma solução é então avaliada pela técnica de recuperação de valor objetivo e passa por um teste de aceitação conforme condição de Boltzmann, onde caso essa solução seja aceita, tanto os parâmetros quanto a solução atual são atualizados e os vetores de histórico das iterações anteriores são recuperados.

Assim que uma solução melhor é encontrada, se atualiza a melhor solução, que após o resfriamento realizado por cada iteração chegar na temperatura absoluta, aceita esta solução.

Algoritmo 7 Simulated Annealing para PM com rotas

1: $s \leftarrow$ solução inicial
2: $s_{melhor} \leftarrow s$
3: $G \leftarrow$ ganho inicial
4: $p \leftarrow$ lucro inicial
5: $f \leftarrow$ tempo de viagem inicial
6: $T \leftarrow T_0$
7: **enquanto** $(T > T_{abs})$ **faça**
8: **para** (u em $tentativas$) **faça**
9: $k \leftarrow$ pegue um item aleatoriamente
10: $\overline{p} \leftarrow p + p_k$
11: **se** ($\overline{p} >$ capacidade da mochila) **então**
12: pule essa iteração
13: **fim se**
14: $\overline{f} \leftarrow$ avalie o tempo usando a técnica de recuperação de valor objetivo
15: $\overline{G} \leftarrow \overline{p} - R * \overline{f}$
16: $\mu \leftarrow$ número aleatório entre 0 e 1
17: $diferença_energia \leftarrow \overline{G} - G$
18: **se** ($diferença_energia > 0$ ou $exp(diferença_energia/T) > \mu$) **então**
19: $G \leftarrow \overline{G}$
20: $p \leftarrow \overline{p}$
21: $f \leftarrow \overline{f}$
22: $s \leftarrow$ aplique um bit-flip em k
23: recupere os vetores do histórico
24: **fim se**
25: **fim para**
26: **se** (melhoria feita) **então**
27: $s_{melhor} \leftarrow$ s
28: **fim se**
29: $T \leftarrow \alpha * T$
30: **fim enquanto**

5 Resultados

Após construção dos 3 solucionadores do PLV, realizamos testes nas três categorias mostradas na seção (4.1), onde cada solucionador executou as instâncias de cada categoria 10 vezes. A partir da coleta de resultados, realizamos uma média de lucro objetivo, tempo de execução e o cálculo de desvio padrão relativo(ou RSD, do termo em inglês *relative standard deviation*) de cada solução.

As tabelas contendo os resultados por categoria podem ser visualizadas abaixo com os melhores valores de objetivo e tempo em evidência:

- Resultados da Categoria 1:

Instancia	CS2SA			CS2B			MA2B		
	Objetivo	rsd	Tempo	Objetivo	rsd	Tempo	Objetivo	rsd	Tempo
eil76	3394.8	2.41%	1.56	2968	0%	**0.1**	**3874.6**	4.49%	61.62
kroA100	4423.7	0.88%	2.86	3892	0%	**0.1**	**4470.2**	0.41%	58.15
ch130	8964.3	0.57%	5.47	7762	0%	**0.01**	**9260.8**	0.6%	131.91
u159	8358.9	1.3%	7.89	7398	0%	**0.01**	**8629.1**	0.37%	195.98
a280	17693.9	0.27%	13.67	16406	0%	**0.02**	**17984.2**	0.8%	482.67
u574	26003.3	1.09%	8.88	25138	0%	**0.04**	**26333.3**	1.05%	600
u724	**49498.9**	0.6%	12.36	46750	0%	**0.08**	48949.2	0.79%	600

- Resultados da Categoria 2:

Instancia	CS2SA			CS2B			MA2B		
	Objetivo	rsd	Tempo	Objetivo	rsd	Tempo	Objetivo	rsd	Tempo
eil76	18753	0%	20.98	21625	0%	**0.01**	**21957.6**	0.93%	89.09
kroA100	39271	0%	34.99	39605	0%	**0.01**	**41384**	1.27%	112.97
ch130	50695	0%	7.68	50942	0%	**0.02**	**59901.1**	1.67%	177.71
u159	58090	0%	11.17	56031	0%	**0.02**	**58948.6**	1.03%	187.68
a280	107696.1	0%	15.85	104366	0%	**0.07**	**108627.8**	0.47%	579.23
u574	**248583.5**	0%	69.51	227416	0%	**0.56**	247881.3	1.2%	600
u724	**309635.4**	0%	80.25	277635	0%	**0.58**	299690.7	0.78%	600

- Resultado da Categoria 3:

Instancia	CS2SA			CS2B			MA2B		
	Objetivo	rsd	Tempo	Objetivo	rsd	Tempo	Objetivo	rsd	Tempo
eil76	87577	0%	4.1	87577	0%	**0.02**	**87767.7**	0.25%	117.7
kroA100	155585	0%	9.61	155585	0%	**0.03**	**155630**	0%	179.38
ch130	197554.8	0%	5.96	193430	0%	**0.05**	**206047.5**	0.95%	299.54
u159	242201	0%	8.58	242278	0%	**0.07**	**244760**	0%	291.86
a280	421711.7	0%	28.82	411715	0%	**0.23**	**424114**	0%	600
u574	953995.8	0%	22.19	924851	0%	**1.71**	**956941**	0%	600
u724	**1191818.6**	0%	34.17	1164115	0%	**2.63**	1165173	0%	600

Todos os algoritmos foram executados em um Core i7-8565U CPU @ 1.80GHz com 16GB RAM DDR3 utilizando Ubuntu 19.04.

Verificando os resultados obtidos, MA2B se sobressaiu em relação a CS2SA e CS2B em 80,95% das vezes, embora tenha consumido muito mais tempo em 100% das vezes.

CS2SA mostrou-se igualmente eficaz a MA2B nas 3 maiores instâncias, porém com o tempo de execução muito menor, chegando a no máximo 13,37% do tempo do consumido por MA2B.

CS2B consumiu uma quantidade de tempo menor em 100% das vezes, o que pode ser justificado por ficar constantemente preso em máximos locais, dado que diferente de MA2B e CS2SA não utiliza de técnicas tão robustas de dispersão no espaço de solução.

6 Conclusão

Nesta pesquisa foram abordados três algoritmos, são eles *CS2SA*, *CS2B* e *MA2B*, aplicados ao problema do ladrão viajante, conforme a análise dos resultados podemos notar algumas características para cada algoritmo.

CS2B é o algoritmo que se destaca pelo tempo de execução, os resultados apresentados não são os melhores em relação a função objetivo, porém sua relação de tempo de execução e proximidade ao maior objetivo obtido para instâncias pequenas o faz atrativo para este tipo de contexto.

CS2SA possui resultados satisfatórios quando comparados com *CS2B* e *MA2B* em relação a tempo, desvio padrão relativo e proximidade do objetivo com a solução que possui o melhor objetivo, o que nos leva a concluir que possui uma boa aderência para contextos onde seja tolerável essa margem de proximidade da solução ótima e um tempo de execução maior

MA2B apresenta os maiores valores para a função objetivo em 80,95% das instâncias testadas, mesmo que muitas vezes alcançando o tempo limite de execução estipulado, isto o torna um bom candidato para contextos onde seja necessário o maior objetivo possível desconsiderando restrições de tempo.

Em estudos futuros, continuar com a aplicação de algoritmos evolucionários para resolução do problema do ladrão viajante parece um caminho promissor, uma vez que os resultados obtidos nos leva a crer que este tipo de algoritmo consegue lidar melhor com contextos mais abstratos por explorar uma maior região de soluções, ainda que necessite de mais tempo e poder de processamento do que os outros algoritmos testados. Nota-se que os resultados obtidos foram inferiores ao *CS2SA* somente quando o tempo limite proposto foi alcançado, para os casos onde o algoritmo chegou ao fim de sua execução antes deste limitador, os resultados foram maiores.

É possível que MA2B alcance resultados melhores do que os obtidos neste estudo quando houver um limite de tempo de execução maior, entretanto essa afirmação necessita de mais investigação para se provar verdadeira.

Referências

AARTS, E. H.; KORST, J. H. Boltzmann machines for travelling salesman problems. *European Journal of Operational Research, 39*, p. 79–95, 1989.

AHRENS, S. A seleção simultânea do Ótimo regime de desbastes e da idade de rotação, para povoamentos de pinus taeda l., através de um modelo de programação dinâmica. p. 1–199, 1992.

ALBRECHT, C. et al. Algoritmos evolutivos aplicados a sÍntese e otimização de sistemas de ancoragem. 2019.

APPLEGATE, D.; COOK, W.; ROHE, A. Chained lin-kernighan for large traveling salesman problems. *INFORMS Journal on Computing*, v. 15, p. 1–119, 2003.

APPLEGATE, D. L. et al. *The Traveling Salesman Problem: A Computational Study*. [S.l.]: Princeton: Princeton University Press, 2007. 608 p. ISBN 978-0-691-12993-8.

BALL, M. . Computational complexity of network reliability analysis: An overview. *IEEE TRANSACTIONS ON RELIABILITY, VOL. R-35, NO. 3*, p. 230–239, 1986.

BELLMAN, R. E. *Dynamic programming (Princeton Landmarks in Mathematics and Physics)*. Softcover. Princeton, N.J ; Woodstock: Princeton University Press, 2010. 392 p. ISBN 9780691146683.

BELLO, I. et al. Neural combinatorial optimization with reinforcement learning. *CoRR*, abs/1611.09940, 2016.

BONYADI, M. R.; MICHALEWICZ, Z.; BARONE, L. The travelling thief problem: The first step in the transition from theoretical problems to realistic problems. *2013 IEEE Congress on Evolutionary Computation*, p. 1037–1044, 2013.

BONYADI, M. R. et al. Socially inspired algorithms for the travelling thief problem. In: *Proceedings of the 2014 Annual Conference on Genetic and Evolutionary Computation*. New York, NY, USA: Association for Computing Machinery, 2014. (GECCO '14), p. 421–428. ISBN 9781450326629. Disponível em: <https://doi.org/10.1145/2576768.2598367>.

BRASSARD, G.; BRATLEY, P. *Fundamentals of Algorithmics*. 1. ed. Nova Iorque: Prentice Hall, 1995. 544 p. ISBN 01-333-5068-1.

BROWNLEE, J. *Clever Algorithms: Nature-Inspired Programming Recipes*. [S.l.: s.n.], 2011. 438 p. ISBN 978-1446785065.

CALDAS, R. B. *PROJETO E ANÁLISE DE ALGORITMOS*. Universidade Federal de Minas Gerais, 2004.

CASTRO, L. de; ZUBEN, F. V. *IA707 – Hill Climbing e Simulated Annealing*. 201–? DCA/FEEC/Unicamp. Disponível em: <ftp://ftp.dca.fee.unicamp.br/pub/docs/vonzuben/ia707_02/topico1_02.pdf>. Acesso em: 12 de mai. de 2019.

CHAN, D.; MERCIER, D. Ic insertion: an application of the travelling salesman problem. *International Journal of Production Research*, p. 27:10, 1837–1841, 1989.

COOK, S. A. The complexity of theorem-proving procedures. *Proceedings of the Third Annual ACM Symposium on Theory of Computing - STOC '71*, p. 151–158, 1971.

CORMEN, T. H. et al. *Algoritmos, teoria e prática*. 3. ed. São Paulo: GEN LTC, 2012. 944 p. ISBN 85-352-3699-6.

CROES, G. A. A method for solving traveling-salesman problems. *Operations Research*, p. 791–812, 1958.

CUNHA, C. B. da. Aspectos práticos da aplicação de modelos de roteirização de veículos a problemas reais. p. 51–74, 2000.

DANTZIG, G. B. Discrete-variable extremum problems. *Operations Research*, v. 5, p. 266–288, 1957.

DAVIS, L. Handbook of genetic algorithms. *Van Nostrand Reinhold Computer Library*, 1991.

DEEP, K.; MEBRAHTU, H. New variations of order crossover for travelling salesman problem. *International Journal of Combinatorial Optimization Problems and Informatics*, p. 2–13, 2011. Disponível em: <https://www.redalyc.org/articulo.oa?id=265219618002>.

DOWNSLAND, K. Some experiments with simulated annealing techniques for packing problems. *European Journal of Operational Research, n. 68*, p. 389–399, 1993.

DYKSTRA, D. P. *Mathematical programming for natural resource management*. 1. ed. Nova Iorque: McGraw-Hill Book Co., 1984. 318 p. ISBN 0070185522.

DZUBERA, J.; WHITLEY, D. Advanced correlation analysis of operators for the traveling salesman problem. In: DAVIDOR, Y.; SCHWEFEL, H.-P.; MÄNNER, R. (Ed.). *Parallel Problem Solving from Nature — PPSN III*. Berlin, Heidelberg: Springer Berlin Heidelberg, 1994. p. 68–77. ISBN 978-3-540-49001-2.

FAULKNER, H. et al. Approximate approaches to the traveling thief problem. In: *Proceedings of the 2015 Annual Conference on Genetic and Evolutionary Computation*. Nova Iorque: ACM, 2015. (GECCO '15), p. 385–392. ISBN 978-1-4503-3472-3.

FOGEL, L. J.; OWENS, A. J.; WALSH, M. J. *Artificial intelligence through simulated evolution*. New York, NY: Wiley, 1966. Disponível em: <https://cds.cern.ch/record/107769>.

FUX, J. Análise de algoritmos sat para resolução de problemas multivalorados. Universidade Federal de Minas Gerais, Belo Horizonte, p. 1–86, 2004.

GABRIEL, P. H. R.; DELBEM, A. C. B. Fundamentos de algoritmos evolutivos. Instituto de Ciências Matemáticas e de Computação da Universidade de São Paulo, 2008.

GALLARDO, J. E.; COTTA, C.; FERNáNDEZ-LEIVA, A. Exact, metaheuristic, and hybrid approaches to multidimensional knapsack problems. In: _____. [S.l.: s.n.], 2008. p. 365 – 383. ISBN 9780470411353.

GLOVER, F. The knapsack problem: Some relations for an improved algorithm. Carnegie inst of tech Pittsburgh pa Graduate School of Industrial Administration, p. 27, 1965.

KELLERER, H.; PFERSCHY, U.; PISINGER, D. *Knapsack Problems*. [S.l.: s.n.], 2004. ISBN 978-3-540-40286-2.

KIRKPATRICK, S.; GELATT, C. D.; VECCHI, M. P. Optimization by simulated annealing. *Science, 220(4598)*, p. 671–680, 1983.

LAARHOVEN, P. J. M. van; AARTS, E. H. L. *Simulated Annealing: Theory and Applications*. 1. ed. Holanda: Dordrecht Springer, 1987. v. 37. 187 p. ISBN 978-94-015-7744-1.

LAPORTE, G. The traveling salesman problem: An overview of exact and approximate algorithms. *European Journal of Operational Research*, p. 59(2):231 – 247, 1992.

LAPORTE, G. A short history of the traveling salesman problem. p. 25, 2006.

LIN, B. L.; SUN, X.; SALOUS, S. Solving travelling salesman problem with an improved hybrid genetic algorithm. *Journal of computer and communications.*, Scientific Research Publishing, v. 4, n. 15, p. 98–106, November 2016.

MEI, Y.; LI, X.; YAO, X. Improving efficiency of heuristics for the large scale traveling thief problem. *SEAL 2014: Simulated Evolution and Learning*, p. 631–643, 2014.

MICHALEWICZ, Z.; FOGEL, D. B. *How to Solve It: Modern Heuristics*. Enlarged 2nd edition. [S.l.]: Springer Science & Business Media, 2013. 584 p.

MOSCATO, P.; COTTA, C. An introduction to memetic algorithms. *Inteligencia Artificial,revista Iberoamericana De Inteligencia Artificial - AEPIA*, v. 7, 07 2003.

MOURA, A.; OLIVEIRA, J. F. Uma Heurística Composta para a Determinação de Rotas para Veículos em Problemas com Janelas Temporais e Entregas e Recolhas. *Investigação Operacional*, scielopt, v. 24, p. 45 – 62, 06 2004. ISSN 0874-5161.

NORMAN, M.; MOSCATO, P. A competitive-cooperative approach to complex combinatorial search. In: *Technical Report Caltech Concurrent Computation Program, Report*. California, USA: expanded version published at the Proceedings of the 20th Informatics and Operations Research Meeting, Buenos Aires (20th JAIIO), Aug. 1991, 1989. p. 3.15–3.29.

OLAZAR, M. R. R. *Algoritmos evolucionários multiobjetivo para alinhamento múltiplo de sequências biológicas*. 116 XV. Monografia (Dissertação) — Universidade Federal do Rio de Janeiro, COPPE, 2007.

OLIVEIRA, I. C. Complexidade computacional e o problema p vs np. *Instituto de Computação UNICAMP*, p. 125, 2010.

OLIVEIRA, M. A.; GUELPELI, M. V. *BLMSumm – Métodos de Busca Local e Metaheurísticas na Sumarização de Textos*. 12 p. — Centro Universitário de Barra Mansa (UBM), 2013.

O'NEIL, M. A.; BURTSCHER, M. Rethinking the parallelization of random-restart hill climbing: A case study in optimizing a 2-opt tsp solver for gpu execution. *GPGPU-8*, p. 99–108, 2015.

PADBERG, M.; GROTSCHEL, M. The travelling salesman problem. *Polyhedral theory*, p. 251–305, 1985.

POLI, R.; LANGDON, W. B.; MCPHEE, N. F. *A Field Guide to Genetic Programming*. Reprint edition. London, England: Lulu Enterprises, 2008. 252 p. ISBN 978-1409200734.

POLYAKOVSKIY, S. et al. A comprehensive benchmark set and heuristics for the traveling thief problem. *Proceedings of the 2014 Annual Conference on Genetic and Evolutionary Computation. ACM*, p. 477–484, 2014.

RAWAT, U. *Introduction to Hill Climbing | Artificial Intelligence*. 2018. Disponível em: <https://www.geeksforgeeks.org/introduction-hill-climbing-artificial-intelligence/>. Acesso em: 16 de mai. de 2019.

ROCHA, A.; DORINI, L. B. Algoritmos gulosos: definições e aplicações. *SP 53*, 2004.

RODRIGUES, M. A. P. Problema do caixeiro viajante: Um algoritmo para resolução de problemas de grande porte baseado em busca local dirigida. *Universidade Federal de Santa Catarina*, Santa Catarina, Brasil, p. 104, 2000.

RUSSELL, S. J.; NORVIG, P. Artificial intelligence : A modern approach. *Malaysia; Pearson Education Limited*, p. 111–114, 2016.

SIMãO, T. D. *Utilização de algoritmos genéticos para otimização de soluções para o timetabling escolar*. 72 p. Monografia — Universidade Federal de Lavras (UFLA), Santa Rita de Caldas, MG, 2013.

SIPSER, M. *Introdução a teoria da computação*. [S.l.]: Cengage Learning, 2005. 488 p. ISBN 978-8522104994.

SöRENSEN, K. Metaheuristics—the metaphor exposed. *International Transactions in Operational Research*, v. 22, n. 1, p. 3–18, 2015.

TRINDADE, . A. R.; OCHI, L. S. Um algoritmo evolutivo hÃbrido para a formaÃ§Ãde cÃde manufatura em sistemas de produÃ§Ã. *Pesquisa Operacional*, scielo, v. 26, p. 255 – 294, 08 2006. ISSN 0101-7438.

TRUONG, T. K.; LI, K.; XU, Y. Chemical reaction optimization with greedy strategy for the 0–1 knapsack problem. *Applied Soft Computing*, v. 13, p. 1774–1780, 2013.

TURING, A. M. On computable numbers, with an application to the entscheidungs problem. p. 230–265, 1936.

VIEIRA, H. A.; MARQUES, R. S.; CASTRO, P. A. L. de. Otimização paramétrica através de busca hill climbing distribuída em sistemas multiagentes. *Computer on the Beach*, p. 161–168, 2015.

WHITLEY, D.; HAINS, D.; HOWE, A. Tunneling between optima: Partition crossover for the traveling salesman problem. In: *Proceedings of the 11th Annual Conference on Genetic and Evolutionary Computation*. New York, NY, USA: Association for Computing Machinery, 2009. (GECCO '09), p. 915–922. ISBN 9781605583259. Disponível em: <https://doi.org/10.1145/1569901.1570026>.

WHITLEY, D.; STARKWEATHER, T.; SHANER, D. The traveling salesman and sequence scheduling: Quality solutions using genetic edge recombination. *Citeseer*, 1991.

WOEGINGER, G. J. Exact algorithms for np-hard problems: A survey. *Lecture Notes in Computer Science*, v. 2570, p. 185–207, 2003.

WRIGHT, S. J. Coordinate descent algorithms. *Mathematical Programming*, p. 3–34, 2015.

WU, J. et al. Exact approaches for the travelling thief problem. In: SHI, Y. et al. (Ed.). *Simulated Evolution and Learning*. Cham: Springer International Publishing, 2017. p. 110–121. ISBN 978-3-319-68759-9.

WUIJTS, R. H. Investigation of the travelling thief problem. Utrecht University, 2018.

YAFRANI, M. E.; AHIOD, B. A local search based approach for solving the travelling thief problem. *Applied Soft Computing*, v. 52, p. 795–804, 2016.

YAFRANI, M. E.; AHIOD, B. Population-based vs. single-solution heuristics for the travelling thief problem. In: *Proceedings of the Genetic and Evolutionary Computation Conference 2016*. New York, NY, USA: Association for Computing Machinery, 2016. (GECCO '16), p. 317–324. ISBN 9781450342063. Disponível em: <https://doi.org/10.1145/2908812.2908847>.

YAFRANI, M. E.; AHIOD, B. Efficiently solving the traveling thief problem using hill climbing and simulated annealing. *Information Sciences*, v. 432, 12 2017.

ZHANG, G. et al. Solving satisfiability problems with membrane algorithms. *Fourth International on Conference on Bio-Inspired Computing*, IEEE, p. 29–36, 2009.